CRUD de Biblioteca

01 Alcance y modelo

La aplicación Django se monta en /biblioteca/. El CRUD web completo corresponde a Libro y Usuario. La portada resume inventario y muestra hasta 15 préstamos recientes. La ruta /biblioteca/crear/ registra préstamos. Ejemplar, Prestamo y Multa también están registrados en el administrador de Django.

```
Libro  1 --- N  Ejemplar  1 --- N  Prestamo  1 --- 0..1  Multa
Cuenta Django  1 --- N  Prestamo
Cuenta Django  1 --- 0..1  Usuario (perfil de Biblioteca)
```

Qué representa cada registro

Libro representa un título. Ejemplar representa una copia física, identificada por un código de barras único. El préstamo referencia la copia física y la cuenta de Django. Usuario guarda el perfil institucional y admite perfiles sin cuenta vinculada.

Cómo viaja una solicitud

sistema/urls.py delega a biblioteca/urls.py. La URL selecciona una función de views.py y le entrega request y, si corresponde, pk. La vista consulta el ORM y devuelve HTML mediante render(). En escrituras, procesa request.POST, valida, guarda, publica un mensaje y redirige cuando la operación se completa.

Las plantillas declaran sus campos HTML directamente. El token CSRF acompaña a los formularios POST. La ruta usa pk como alias de la clave primaria, aunque cada modelo tenga un nombre de columna propio. Un identificador inexistente en las vistas de detalle, edición y borrado responde 404.

CRUD de Biblioteca

02 CRUD de libros

Prefijo /biblioteca/libros/. El listado y el detalle son públicos. Las otras tres rutas requieren una sesión Django, tanto al abrir el formulario como al enviarlo.

```
GET       libros/                 libro_listar
GET       libros/<pk>/            libro_detalle
GET/POST  libros/crear/           libro_crear
GET/POST  libros/<pk>/editar/     libro_actualizar
GET/POST  libros/<pk>/eliminar/   libro_eliminar
```

Leer el catálogo

El listado consulta todos los libros y los ordena por título. El detalle recupera el libro por pk y lista sus ejemplares ordenados por código de barras. Las plantillas son libro_list.html y libro_detail.html.

Crear y actualizar

Crear instancia Libro() y editar recupera el registro existente. Ambas vistas usan libro_form.html y _validar_libro(). El helper recorta espacios, convierte opcionales vacíos a None y llama a full_clean(). Los errores se muestran por campo y se conserva el POST para corregirlo. Si la validación pasa, save() persiste y la vista redirige al detalle.

Título obligatorio (120 caracteres). Autor (100), ISBN (17), editorial (60) y año son opcionales. El ISBN debe ser único cuando está informado. El año se valida como entero. El modelo carece de un rango de años y de una validación de estructura o dígito de control del ISBN.

Eliminar

GET muestra libro_confirm_delete.html. POST comprueba si hay ejemplares y, si existen, vuelve al detalle con un mensaje. Si el libro está libre de ejemplares, lo borra y vuelve al listado. La clave foránea usa PROTECT como segunda barrera.

CRUD de Biblioteca

03 CRUD de usuarios

Prefijo /biblioteca/usuarios/. La consulta pública muestra datos institucionales. Las altas, ediciones y bajas requieren sesión.

```
GET       usuarios/                 usuario_listar
GET       usuarios/<pk>/            usuario_detalle
GET/POST  usuarios/crear/           usuario_crear
GET/POST  usuarios/<pk>/editar/     usuario_actualizar
GET/POST  usuarios/<pk>/eliminar/   usuario_eliminar
```

Lectura

El listado ordena por id_usuario descendente y trae la cuenta asociada con select_related(). El detalle consulta el historial de préstamos usando la cuenta vinculada y ordena por fecha descendente. Los préstamos se guardan contra la cuenta Django, así que un perfil sin cuenta carece de historial asociado.

Alta y edición

RUT, nombre y correo son obligatorios. Las vistas verifican RUT y correo duplicados, excluyendo el registro actual al editar. Departamento es opcional. El checkbox se interpreta como verdadero si llega activo=on. Estas vistas hacen validación manual y omiten full_clean(), incluida la validación de correo en el servidor.

El alta deriva el username de la parte local del correo, cambia puntos por guiones bajos y añade un sufijo si detecta una colisión. Crea primero la cuenta y después el perfil. La edición sincroniza nombres, correo e is_active cuando hay cuenta, y conserva el username. Ambas redirigen al listado.

Baja e integridad

POST bloquea la baja si hay préstamos ACTIVO. Si solo hay historial en otro estado, PROTECT puede impedir borrar la cuenta después de que el perfil ya se haya borrado. Los cambios de cuenta y perfil se realizan en escrituras separadas. Una corrección debe decidir si conserva el historial mediante desactivación y agrupar los cambios en una transacción.

CRUD de Biblioteca

04 Préstamos e integridad

Registro actual

prestamo_crear presenta ejemplares DISPONIBLE y cuentas activas. Al enviar el formulario, busca ambos objetos por pk, convierte la fecha ISO y los días, calcula la devolución pactada y crea un préstamo ACTIVO. Luego marca la copia PRESTADO y redirige a la portada.

Reglas de eliminación del modelo

PROTECT impide borrar un libro referenciado por un ejemplar y un ejemplar o cuenta referenciados por préstamos de cualquier estado. La relación Usuario.user usa CASCADE al borrar la cuenta. La multa depende del préstamo con OneToOneField y CASCADE al eliminarlo.

Casos de fallo identificados en el código

Préstamos. Las opciones del formulario filtran por disponibilidad y actividad, pero el POST consulta por pk sin repetir esos filtros. Un envío manipulado puede seleccionar una copia prestada o una cuenta inactiva. Un número de días o una fecha mal formados pueden producir ValueError. Creación y cambio de estado se hacen por separado.

Usuarios. Un fallo en la segunda escritura de un alta o edición puede dejar datos parciales. El sufijo de username se calcula con un conteo y puede colisionar. Las comprobaciones previas de unicidad también pueden competir entre solicitudes.

Libros. Si se crea un ejemplar entre la comprobación de dependencias y el borrado, PROTECT puede elevar ProtectedError. Si dos solicitudes guardan simultáneamente el mismo ISBN, la restricción de base de datos puede elevar IntegrityError.

Pruebas de regresión sugeridas

Probar la baja de un perfil con préstamos históricos, dos solicitudes sobre la misma copia y POST con fechas inválidas. Verificar el estado persistido tras el fallo, además del mensaje. Para un préstamo, validar actividad y disponibilidad al procesar el POST y diseñar una reserva atómica compatible con el motor de base de datos.

CRUD de Biblioteca

05 Verificación reproducible

Se ejecutaron las 16 pruebas existentes de Biblioteca en la base de pruebas de Django. Todas pasaron. La suite cubre lectura de libros, alta válida, título ausente, ISBN duplicado, edición, borrado libre y borrado bloqueado por ejemplares. También cubre login, logout y redirecciones de rutas protegidas.

```
.\venv\Scripts\python.exe manage.py test biblioteca
```

Comprobación manual con datos de prueba

1. Abrir /biblioteca/libros/ sin sesión y entrar al detalle de un libro. Abrir /biblioteca/libros/crear/ y comprobar el salto al login con next.

2. Iniciar sesión con una cuenta de prueba. Crear un libro sin ISBN. Repetir con un ISBN que ya exista y verificar que el formulario conserve los datos y muestre el error.

3. Editar el título y comprobar el cambio en la ficha. Abrir el diálogo de borrado y comprobar que GET no cambia la base. Enviar la confirmación POST con un libro sin ejemplares y repetir con otro que tenga ejemplares.

4. Crear un perfil con RUT y correo de prueba. Editar el correo y el estado activo y revisar la cuenta asociada. Probar la baja de un perfil sin préstamos y la de otro con un préstamo activo.

Archivos de referencia

biblioteca/models.py · entidades y relaciones. biblioteca/views.py · validación, persistencia y respuesta. biblioteca/urls.py y sistema/urls.py · montaje de rutas. biblioteca/templates/biblioteca/ · pantallas y formularios. biblioteca/admin.py · administración. biblioteca/tests.py · cobertura existente.

CRUD de Biblioteca

01 Conectar las rutas

El proyecto monta la aplicación bajo /biblioteca/. Django entrega pk a la vista cuando la ruta captura un entero. Estos son los patrones existentes para libros.

```
# sistema/urls.py (dentro de urlpatterns)
path("biblioteca/", include("biblioteca.urls")),

# biblioteca/urls.py (dentro de urlpatterns)
path("libros/", views.libro_listar, name="libro_listar"),
path("libros/crear/", views.libro_crear, name="libro_crear"),
path("libros/<int:pk>/", views.libro_detalle,
     name="libro_detalle"),
path("libros/<int:pk>/editar/", views.libro_actualizar,
     name="libro_actualizar"),
path("libros/<int:pk>/eliminar/", views.libro_eliminar,
     name="libro_eliminar"),
```

En una plantilla, {% url 'libro_detalle' libro.pk %} construye la URL con el identificador. Por ejemplo, pk=7 produce /biblioteca/libros/7/. El nombre de ruta también sirve en redirect() y reverse().

CRUD de Biblioteca

02 Crear un libro desde HTML

Ejemplo de libro_form.html que conserva los cinco campos del proyecto. Al omitir action, el formulario envía el POST a la URL actual. Se puede reutilizar para alta y edición.

```
<h1>{{ accion }} libro</h1>
<form method="post" novalidate>
  {% csrf_token %}
  <label for="titulo">Título</label>
  <input id="titulo" name="titulo" maxlength="120"
         value="{{ datos.titulo|default_if_none:'' }}" required>
  {% for error in errores.titulo %}<p>{{ error }}</p>{% endfor %}

  <label for="autor">Autor</label>
  <input id="autor" name="autor" maxlength="100"
         value="{{ datos.autor|default_if_none:'' }}">
  {% for error in errores.autor %}<p>{{ error }}</p>{% endfor %}

  <label for="isbn">ISBN</label>
  <input id="isbn" name="isbn" maxlength="17"
         value="{{ datos.isbn|default_if_none:'' }}">
  {% for error in errores.isbn %}<p>{{ error }}</p>{% endfor %}

  <label for="editorial">Editorial</label>
  <input id="editorial" name="editorial" maxlength="60"
         value="{{ datos.editorial|default_if_none:'' }}">
  {% for error in errores.editorial %}<p>{{ error }}</p>{% endfor %}

  <label for="anio">Año</label>
  <input id="anio" name="anio_publicacion" type="number"
         value="{{ datos.anio_publicacion|default_if_none:'' }}">
  {% for error in errores.anio_publicacion %}
    <p>{{ error }}</p>
  {% endfor %}
  <button type="submit">Guardar</button>
</form>
```

label se asocia al id para identificar el control. name es la clave que llega al servidor, por ejemplo request.POST.get("anio_publicacion"). novalidate deja que Django muestre los errores del servidor en este ejemplo.

CRUD de Biblioteca

03 Del POST a la base de datos

Vista de alta simplificada. Reutiliza _validar_libro() del proyecto y omite únicamente el mensaje de éxito de la vista original.

```
@login_required
def libro_crear(request):
    datos = {}
    errores = {}
    if request.method == "POST":
        datos = request.POST
        libro = Libro()
        errores = _validar_libro(libro, datos)
        if not errores:
            libro.save()
            return redirect("libro_detalle", pk=libro.pk)

    return render(request, "biblioteca/libro_form.html", {
        "datos": datos,
        "errores": errores,
        "accion": "Crear",
    })
```

Si llega titulo=" El principito ", el helper guarda "El principito". Si ISBN llega vacío, se convierte en None. full_clean() valida el tipo del año, las longitudes y la unicidad del ISBN. Cuando detecta errores, la vista conserva los datos y vuelve al formulario. Un POST válido guarda y responde con una redirección al detalle.

CRUD de Biblioteca

04 Leer y enlazar el detalle

La vista de listado entrega libros. La vista de detalle entrega libro y ejemplares. Este extracto de HTML muestra cómo se consumen esos nombres.

```
<!-- libro_list.html -->
<h1>Libros</h1>
<ul>
  {% for libro in libros %}
    <li><a href="{% url 'libro_detalle' libro.pk %}">
      {{ libro.titulo }}
    </a></li>
  {% empty %}
    <li>Todavía no hay libros.</li>
  {% endfor %}
</ul>

<!-- libro_detail.html -->
<h1>{{ libro.titulo }}</h1>
<p>Autor: {{ libro.autor|default:"Sin registrar" }}</p>
<h2>Ejemplares</h2>
<ul>
  {% for ejemplar in ejemplares %}
    <li>{{ ejemplar.codigo_barra }} - {{ ejemplar.estado }}</li>
  {% empty %}
    <li>Este libro aún no tiene ejemplares.</li>
  {% endfor %}
</ul>
<a href="{% url 'libro_actualizar' libro.pk %}">Editar</a>
<a href="{% url 'libro_eliminar' libro.pk %}">Eliminar</a>
```

Los enlaces disparan GET. Abrir «Eliminar» lleva a la pantalla de confirmación. El borrado ocurre al enviar el formulario POST de esa pantalla.

CRUD de Biblioteca

05 Editar y confirmar el borrado

Edición abre el formulario compartido con datos del registro. Este ejemplo reutiliza el helper original y omite el mensaje de éxito para centrarse en el flujo.

```
@login_required
def libro_actualizar(request, pk):
    libro = get_object_or_404(Libro, pk=pk)
    datos = {
        "titulo": libro.titulo, "autor": libro.autor,
        "isbn": libro.isbn, "editorial": libro.editorial,
        "anio_publicacion": libro.anio_publicacion,
    }
    errores = {}
    if request.method == "POST":
        datos = request.POST
        errores = _validar_libro(libro, datos)
        if not errores:
            libro.save()
            return redirect("libro_detalle", pk=libro.pk)
    return render(request, "biblioteca/libro_form.html", {
        "datos": datos, "errores": errores,
        "accion": "Editar", "libro": libro,
    })
```

```
<!-- libro_confirm_delete.html -->
<h1>Eliminar {{ libro.titulo }}</h1>
<form method="post">
  {% csrf_token %}
  <button type="submit">Confirmar eliminación</button>
</form>
<a href="{% url 'libro_detalle' libro.pk %}">Cancelar</a>
```

En la vista libro_eliminar, GET renderiza esta confirmación. POST comprueba Ejemplar.objects.filter(id_libro=libro).exists(). Si hay copias, vuelve a la ficha con un mensaje. Si no hay, ejecuta libro.delete() y redirige al listado.

CRUD de Biblioteca

06 Borrar el libro en la vista

Versión simplificada de libro_eliminar. Usa el mismo control de acceso y la misma comprobación de ejemplares que el proyecto.

```
@login_required
def libro_eliminar(request, pk):
    libro = get_object_or_404(Libro, pk=pk)
    if request.method == "POST":
        if Ejemplar.objects.filter(id_libro=libro).exists():
            messages.error(request, "El libro tiene ejemplares.")
            return redirect("libro_detalle", pk=libro.pk)

        libro.delete()
        messages.success(request, "Libro eliminado.")
        return redirect("libro_listar")

    return render(
        request,
        "biblioteca/libro_confirm_delete.html",
        {"libro": libro},
    )

<!-- En libro_detail.html o libro_list.html -->
{% for mensaje in messages %}
  <p>{{ mensaje }}</p>
{% endfor %}
```

Ejemplo con pk=7. GET /biblioteca/libros/7/eliminar/ muestra la confirmación. El botón envía POST a esa misma URL. Si hay ejemplares, el detalle muestra el mensaje y el libro sigue en la base. Si se borra, el listado muestra el mensaje de éxito.

CRUD de Biblioteca

07 Formulario del perfil institucional

Ejemplo mínimo para usuario_form.html. El alta toma los valores de datos al devolver un error. La edición abre con los valores del perfil usuario, tal como hace la vista actual.

```
<h1>{{ accion }} usuario</h1>
{% for mensaje in messages %}<p>{{ mensaje }}</p>{% endfor %}
{% firstof usuario.rut datos.rut "" as rut %}
{% firstof usuario.nombre_completo datos.nombre_completo "" as nombre %}
{% firstof usuario.email_institucional datos.email_institucional "" as correo %}
{% firstof usuario.carrera_o_departamento datos.carrera_o_departamento "" as depto %}
<form method="post">
  {% csrf_token %}
  <label for="rut">RUT</label>
  <input id="rut" name="rut" maxlength="12" required
         value="{{ rut }}">
  <label for="nombre">Nombre completo</label>
  <input id="nombre" name="nombre_completo" required
         maxlength="100"
         value="{{ nombre }}">
  <label for="correo">Correo institucional</label>
  <input id="correo" name="email_institucional" type="email"
         maxlength="100" required
         value="{{ correo }}">
  <label for="departamento">Departamento</label>
  <input id="departamento" name="carrera_o_departamento"
         maxlength="100"
         value="{{ depto }}">
  <label for="activo">Activo</label>
  <input id="activo" name="activo" type="checkbox"
         {% if usuario.activo or datos.activo %}checked{% endif %}>
  <button type="submit">Guardar</button>
</form>
```

Cuando la casilla está marcada, el navegador envía activo=on. Si no está marcada, omite la clave. El servidor convierte el valor con request.POST.get("activo") == "on". El tipo email ayuda en el navegador. La validación actual del servidor se describe en el informe.